🌲 DAY 3: MASTER ARROW WORKFLOWS & COMPONENT EXECUTION TREES

Deep Learning on HPC with TensorFlow, Slurm & GPUs

Course Code: `GLA-HPC-BCSG701` | **Track:** Intel & NEC Supercomputing Alliance
Lead Trainer: Shivam Gupta | **Target Session:** Day 3 (Strictly 3 Hours / 180 Minutes)
Primary Reference: `Intel & NEC - High Performance Computing.pdf` (Page 2)

🗺️ MASTER INDEX OF DAY 3 ARROW TREES

- Tree 1: Deep Learning Pipeline & I/O Prefetching Architecture Flow
- Tree 2: Intel oneDNN Operator Fusion & Memory Wall Elimination Flow
- Tree 3: Deep Learning Accelerators: Xeon AMX vs GPU Max vs Habana Gaudi Flow
- Tree 4: Slurm Workload Manager Job Lifecycle Flow
- Tree 5: Horovod Ring-AllReduce Gradient Synchronization Tree
- Tree 6: The Complete Multi-Node Distributed Training Lifecycle

Tree 1: Deep Learning Pipeline & I/O Prefetching Architecture Flow

```
[ Raw ImageNet / Video Dataset on Lustre Parallel Storage ]
       │
       ▼
[ Storage Read over High-Speed Fabric ] ──► (Bottleneck Risk: Storage I/O)
       │
       ▼
[ CPU Multi-Core Decoding & Augmentation Pipeline ]
       ├── `tf.data.Dataset.map(decode_fn, num_parallel_calls=tf.data.AUTOTUNE)`
       └── Workers decode JPEG/PNG into raw Float32 tensors across Xeon cores
       │
       ▼
[ Asynchronous Buffer Prefetching ]
       └── `dataset.prefetch(buffer_size=tf.data.AUTOTUNE)`
       │   (CPU actively loads Batch N + 1 into RAM while Accelerators compute Batch N!)
       │
       ▼
[ PCIe Gen 5 / CXL DMA Stream ] ──► Direct Transfer to GPU / Accelerator Memory
       │
       ▼
[ Hardware Accelerators Saturated at 95%+ Continuous Utilization! ]
```

Tree 2: Intel oneDNN Operator Fusion & Memory Wall Elimination Flow

```
[ Standard Deep Learning Layer Sequence ]
Input Tensor ──► [ Conv2D ] ──► [ Bias Add ] ──► [ ReLU Activation ] ──► Output Tensor
       │
       ├── WITHOUT Operator Fusion (Traditional Eigen/TensorFlow Engine):
       │     1. Conv2D reads from DRAM ──► Computes ──► Writes intermediate to DRAM
       │     2. Bias Add reads from DRAM ──► Adds bias ──► Writes intermediate to DRAM
       │     3. ReLU reads from DRAM ──► Clamps negatives ──► Writes final to DRAM
       │     (Result: 3 Heavy Round-Trips to Main Memory = Bandwidth Exhaustion!)
       │
       └── WITH Intel oneDNN Hardware Operator Fusion:
             │
             ▼
      [ Fused oneDNN Execution Kernel ]
             ├── Input streamed directly into Xeon AVX-512 / AMX Tile Registers
             ├── Convolution evaluated in high-speed hardware accumulator
             ├── Bias added while data is still HOT in CPU L1 Cache / Register
             └── ReLU activation clamped instantly before writing to memory
             │
             ▼
      [ Result: ONE Single Memory Pass = 2.5× to 3.2× Throughput Speedup! ]
```

Tree 3: Deep Learning Accelerators: Xeon AMX vs GPU Max vs Habana Gaudi Flow

```
                [ Deep Learning Compute Workload ]
                                 │
        ┌────────────────────────┼────────────────────────┐
        ▼                        ▼                        ▼
[ Intel Xeon CPU ]     [ Intel GPU Max 1550 ]   [ Habana Gaudi 2 & 3 ]
(4th/5th Gen Scalable)   (Ponte Vecchio)          (Purpose-Built AI ASIC)
        │                        │                        │
        ▼                        ▼                        ▼
• Host control & I/O     • 128 Xe-cores           • Dedicated Matrix Engines (MME)
• Intel AMX Silicon      • Hardware XMX Systolic  • Programmable TPC VLIW Cores
  (Tile Registers:         Engines (BF16/INT8)    • 96 GB / 128 GB HBM2e
  tmm0-tmm7, 8 KB)       • 128 GB HBM2e           • 24x 100/200GbE RoCE v2 Ports
• 1024 BF16 ops/cycle    • 3.2 TB/s Bandwidth       INTEGRATED DIRECTLY ON DIE!
• Native FP32/BF16/INT8  • XeLink Bridges for     • Direct all-to-all cluster
  inference throughput     direct GPU-to-GPU fabric  fabric without external NICs!
```

Tree 4: Slurm Workload Manager Job Lifecycle Flow

```
[ Engineer Writes Batch Job Script: `train_resnet_hpc.slurm` ]
       |
       ▼
[ Submission Command: `sbatch train_resnet_hpc.slurm` ]
       |
       ▼
[ Slurm Controller (`slurmctld`) on Master / Head Node ]
       ├── 1. Validates script syntax, account limits, and walltime
       ├── 2. Assigns unique Job ID (e.g., Job 104821)
       └── 3. Enters Cluster Priority Queue in state: PENDING (PD)
       |
       ▼ (Resources become available across cluster)
[ Resource Allocation & Dispatch ]
       ├── `slurmctld` allocates 2 Compute Nodes + 8 Tasks + 64 Cores
       └── State transitions: PENDING (PD) ──► RUNNING (R)
       |
       ▼
[ Node Execution Daemon (`slurmd`) on Compute Nodes ]
       ├── Spawns local worker processes via `srun` / `mpirun`
       ├── Pins threads to assigned hardware cores
       └── Streams console stdout/stderr directly into `/scratch/logs/104821.out`
       |
       ▼
[ Job Completion & Teardown ]
       ├── Node state transitions: ALLOCATED ──► IDLE
       └── Exit code returned: `exit 0` (SUCCESS) or non-zero (FAILED)
```

Tree 5: Horovod Ring-AllReduce Gradient Synchronization Tree

```
[ 4 Data-Parallel Worker Nodes (Ranks 0, 1, 2, 3) ]
Each node computes forward/backward pass on its local mini-batch
       |
       ▼
[ Local Gradients Evaluated: G0, G1, G2, G3 ]
Each gradient vector is split into 4 equal chunks: [a, b, c, d]
       |
       ▼
[ PHASE 1: SCATTER-REDUCE (Ring Transfer: 0 ──► 1 ──► 2 ──► 3 ──► 0) ]
  • Step 1: Rank 0 sends chunk [a] to Rank 1; Rank 1 adds local [a]
  • Step 2: Rank 1 sends reduced [a] to Rank 2; Rank 2 adds local [a]
  • Step 3: Rank 2 sends reduced [a] to Rank 3; Rank 3 holds FULL SUM of chunk [a]!
  (Concurrently: Rank 0 holds sum of [b], Rank 1 holds sum of [c], Rank 2 holds sum of [d])
       |
       ▼
[ PHASE 2: ALLGATHER (Circulating Reduced Chunks around Ring) ]
  • Step 1: Fully reduced chunks are forwarded to adjacent neighbors
  • Step 2: Slices circulate until every rank holds chunks [a + b + c + d]
       |
       ▼
[ IDENTICAL GRADIENT APPLIED AT EVERY WORKER NODE ]
Total Communication Overhead: Independent of node count $N$!
Result: Linear 90%+ Scaling Efficiency!
```

Tree 6: The Complete Multi-Node Distributed Training Lifecycle

```
[ 1. PROBLEM FORMULATION & DATASET STAGING ]
ImageNet / Parquet dataset staged in high-speed parallel `/scratch` storage
       |
       ▼
[ 2. HOROVOD INTEGRATION ]
Wrap model optimizer with `hvd.DistributedOptimizer()` ──► Apply linear LR scale
       |
       ▼
[ 3. SLURM BATCH SCRIPT CONFIGURATION ]
Specify `#SBATCH --nodes=4 --ntasks-per-node=4 --cpus-per-task=8 --time=04:00:00`
       |
       ▼
[ 4. QUEUE SUBMISSION & MONITORING ]
`sbatch train.slurm` ──► Monitor queue via `squeue -u $USER`
       |
       ▼
[ 5. DISTRIBUTED EXECUTION ]
`mpirun` launches 16 workers ──► Ring-AllReduce synchronizes gradients over InfiniBand
       |
       ▼
[ 6. CHECKPOINTING & LOGGING ]
Rank 0 exclusively saves model checkpoints to `/scratch/checkpoints/`
       |
       ▼
[ 7. METRIC & EFFICIENCY AUDIT ]
Compute aggregate Images/Sec throughput ──► Verify $\ge 85\%$ multi-node scaling efficiency
```

Workflows Status: 100% Complete & Verified | GLA University HPC Masterclass Day 3